Repeat Purchase Prediction

Pipeline Structure and Rationale

Group 22 • Machine Learning, NOVA IMS
dataset “Everything & Then Some”

Handout PDF

Members
Mohamed Triki • Wael Farhat • Gustavo Carvalho

Kaggle team ML_Group_22

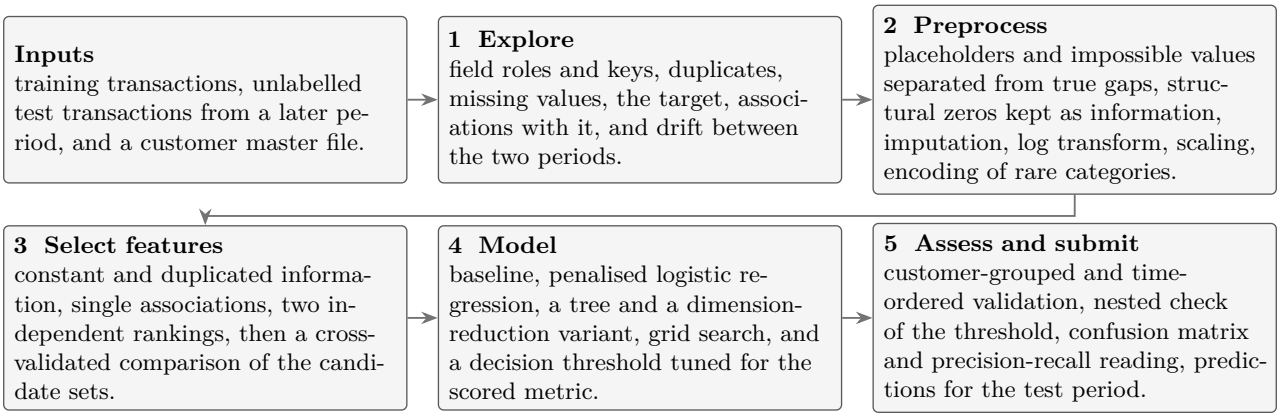
1. Objective and data

The task is to predict whether the customer behind a purchase will buy again within ninety days. The Kaggle competition scores the positive class with F1, so the work is judged on how reliably repeaters are found, not on how often the model is right overall.

Training covers 6,534 transactions from 610 customers between July 2015 and February 2025; the test file covers 986 transactions from 206 customers between May 2025 and October 2026 and carries no label. A customer file describes 1,143 accounts and joins on the customer key. The two transaction files share customers but not rows, and 94.5% of test rows belong to a customer the company has already seen, which shapes how we validate. After cleaning, 56% of transactions repeat, so the classes are close in size and always answering yes already scores 0.718.

2. Structure of the pipeline

Four notebooks (stages 4 and 5 share one), each handing its output to the next as a plain table.



3. The stages and why each one suits this data

Preprocessing. Some amount fields held placeholders for 'unknown' or impossible negative values. These 88 cells (35 placeholders, 53 negatives) became missing values, flagged rather than clipped. A further 18 negative mean purchase values (16 train, 2 test) were missed. Their totals are positive, so they look like sign errors. They were left unchanged. Replacing them with the column median on a copy moves grouped F1 by less than 0.001. The 65 rows with every transaction field empty were dropped, leaving 6,469. A first or second purchase has no history to measure: 1,048 training rows come from customers with zero or one earlier order, so the fields that cannot be measured were set to zero and marked. Amounts are skewed, so a log transform and scaling follow, and rare categories were grouped before encoding.

Integrating the customer data. The customer file is optional, and the key was checked first so no transaction is duplicated. Region and join year differ in repeat rate, but their values are fixed per customer, so they can work as hidden IDs. One region repeats on 94% of its rows, yet one customer supplies 86% of that region's rows. Dropping all 33 customer-file columns moves grouped F1 only from 0.7484 to 0.7481, so they add nothing measurable but do no harm. Street, company name and postal code were left out because they identify a customer rather than describe behaviour.

Feature selection. Selection was made by concept rather than by column, because encoding one category as several indicators makes those columns dependent. Constant and near-duplicate fields go first, then weak single associations, then two separate rankings, and finally a cross-validated comparison of the candidate sets. The funnel runs from 53 concepts and 124 columns to 19 and 64. The grouped folds put both within noise (0.741 against 0.748), so the model keeps all 124 columns.

Modelling. With a few thousand rows of tabular, business-readable fields, a penalised logistic regression is the natural first choice: stable at this size, lightly tuned, and one interpretable coefficient per feature. A decision tree and a dimension-reduction variant were fitted as checks beside an always-yes baseline; the linear model won, 0.748 against 0.723 and 0.745 (grouped folds at 0.50, best set per model; Figure 1).

Assessment. The same customers appear many times, so a random split tests the model on rows whose customer already helped fit it; the folds are grouped on the customer. Because the score depends

on a decision threshold as much as on the model, that threshold was tuned on out-of-fold probabilities and re-verified inside a nested loop. A final check fits the first 80% and scores the last 20%.

4. Results

The grouped 0.755 (threshold tuned inside each fold; 0.748 at a fixed 0.50) is the stricter estimate, since those folds use customers the model has never met. The time-ordered 0.795 looks higher, but the later period has more repeaters, so always-yes already scores 0.754 there. The gain over always-yes is about 0.04 in both checks. The submission scores 0.782.

Measure	Model F1	Always-yes F1
Customer-grouped CV, nested threshold	0.755	0.718
Time-ordered check, all rows	0.795	0.754
returning customers	0.801	0.762
first appearance	0.730	0.688
Kaggle, public part of the test set	0.782	–

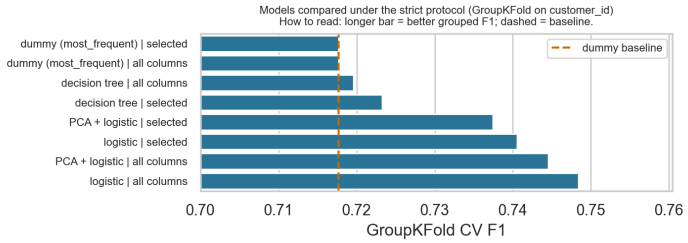


Figure 1. Every model family under the customer-grouped protocol, against the baseline. The best model uses all 124 columns; the 64 selected columns score 0.741 (fold sd 0.0095).

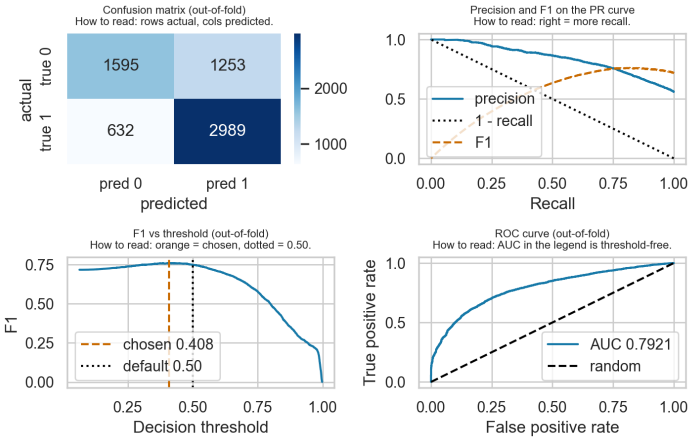


Figure 2. Out-of-fold results at the chosen threshold of 0.408: confusion matrix, precision-recall trade, F1 against threshold, and ROC curve (AUC 0.792).

Annexes

A. Reproducing the work. Four notebooks, fixed seed, about two minutes. Only course-taught tools were used: no resampling, bagging or boosting.

B. Choices that were tested and then dropped. Customer-level history features, ratios, winsorising, quartile binning and interactions were all tried; none beat the noise between folds, and the customer aggregates made things worse. Scaling the yes-no flags as well as the amounts was also worse.

C. Limitations. The threshold (0.408) was chosen on the training period. It flags 65.6% of out-of-fold predictions as repeaters against 56% real, and 76.5% of test rows, so it is the first value to revisit if the private leaderboard disagrees. The transformations were fitted once on all training rows before cross-validation; only the model is re-fitted inside each fold. The time-ordered check reuses a subset of the training rows, so the threshold, the regularisation strength and the transformations were all chosen with the holdout included.